ChE 205 – Computational Methods in Chemical Engineering – Fall 2019
Student Learning Outcome 1
Rubric for assessment problem on final exam (Problem # 1c, 2, 3a, and 4)
Application of computational methods in solving chemical engineering problems.

SLO 1 Performance Indicator (PI)	**(1)** Unsatisfactory	**(2)** Developing	**(3)** Satisfactory
1. Apply mathematical and scientific principles to formulate models and systems relevant to the subject Problem 1c, 7 pts	Able to successfully combines mathematical and/or scientific principles to formulate models and systems relevant to chemical engineering (<3 pts)	Chooses a mathematical model or scientific principle that applies to an engineering problem, but has trouble in model development (3-5 pts)	Does not understand the connection between mathematical models and the system or process to be analyzed or designed (6-7 pts)
2. Solve engineering problems by using the concepts of algebra, calculus, differential equations and/or linear algebra Problem 2, 10 pts	applies concepts of algebra, calculus, differential equations and/or linear algebra to solve chemical engineering problems (<4 pts)	Shows nearly complete understanding of applications of calculus and/or linear algebra in problem-solving (4-7 pts)	Does not understand the application of algebra, calculus, differential equations and/or linear algebra in solving chemical engineering problems (8-10 pts)
4. Translates academic theory into engineering applications Problem 4, 5 pts	Translates academic theory into engineering applications and accepts limitations of mathematical models of physical reality (< 2 pts)	Some gaps in understanding the application of theory to the problem and expects theory to predict reality (2-3 pts)	Does not appear to grasp the connection between theory and the problem (4-5 pts)
6. The uses of appropriate resources needed to solve problems Problem 3a, 3 pts	Uses no resources to solve problems (< 2 pts)	Uses limited resources to solve problems (2 pts)	Uses appropriate resources to locate information needed to solve problems (3 pts)

Averages:

Performance Indicators: (1) 2.53 **(2)** 1.36 **(4)** 2.51 **(6)** 2.42
Overall Average: 2.21
Standard Deviation: 0.49

Average scores were fairly uniform across Performance Indicators (PI) 1, 2, and 6. A rather low score was observed in PI 2, designated for problem 2. Also, there was wide variation of PI among students. This data suggests that most students were strong in (i) applying mathematical and scientific principles to formulate models, (ii) translate

academic theory into engineering applications, and (iii) uses of appropriate resources needed to solve problems; however they were weak in solving engineering problems using differential equations and linear algebra (31/45 were unsatisfactory to solve problem 2).

Approach 1: To improve the course, several engineering problems focusing on differential equations and linear algebra within the scope of ChE 205 will be provided on a weekly basis, and appropriate approaches to solve them numerically will be discussed.
Approach 2: Math 210 (vector space, linear algebra, differential equations) can be made a prerequisite of ChE 205. Currently, Math 210 can be taken concurrently.

Problem 1 (10 pts). Consider the following nonlinear equation:

$$\left(a + \frac{b}{x}\right)^{\blacklozenge} = \frac{d}{(x - c)}, \tag{1}$$

where $x \geq 1$, $a = 10$, $b = 4.238$, $c = 0.038$, and $d = 42.917$. Find x for which equation 1 is satisfied, using the following steps:

(a) Plot the function whose root you are trying to find. (2 pts)

(b) Based on your results in part a, make a good initial guess, x_0. (1 pt)

(c) Write a VBA script to find the root of the function in part a, using Newton's method, and the initial guess, x_0, you made in part b. The VBA script should take a, b, c, d, x_0, and the tolerance, tol, as inputs through the spreadsheet. Then it should output

i. the solution, x,

ii. the total number of iterations, and

iii. the error at each iteration, as shown below. (7 pts)

	A	B	C	D	E	F
1	10	=a			iteration	error
2	4.238	=b			1	???
3	0.038	=c			2	???
4	42.917	=d			3	???
5	1.00E-09	=tolerance			4	???
6	???	=x0			5	???
7	???	=x			6	???
8	???	=iterations			7	???
9					8	???
10						

Figure 1: Sample input and output of VBA script in Problem 1.

Problem 2 (10 pts). Using the Crank-Nicholson method, write a VBA program to solve the following diffusion equation:

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} - B\frac{\partial u}{\partial x}, \tag{2}$$

subject to the following initial condition:

$$u(x, t = 0) = \sin(\pi x), \tag{3}$$

and boundary conditions:

$$u(x, t) = 0 \text{ at } x = 0 \tag{4}$$

$$\frac{\partial u(x, t)}{\partial x} = 1 \text{ at } x = 1, \tag{5}$$

Set $B = 1$, and use t_{max}=1, Δx=0.05, and Δt=0.001 to plot $u(x, t)$ against $0 \leq x \leq 1$ for four different values of t=0 (blue), 0.01 (green), 0.1 (red), and 1 (black).

Problem 3 (10 pts). A model for a batch reactor is given by:

$$\begin{aligned} \frac{dy}{dt} &= b_1 y - \frac{b_1}{b_2} y^2, \quad y(t=0) = 0.03 \\ \frac{dz}{dt} &= b_3 y, \qquad\qquad z(t=0) = 0.0 \end{aligned}$$

where

y = dimensionless concentration of the first component

z = dimensionless concentration of the second component

t = dimensionless time, $0 \leq t \leq 1$.

Experiments have determined that $b_1 = 13.1$, $b_2 = 0.94$, and $b_3 = 1.71$.

(a) Solve this ordinary differential equation - initial value problem (ODE-IVP) using forward Euler in spreadsheet (this is not VBA programming) with $\Delta t = 0.005$. Plot y and z in red and blue, respectively, vs $0 \leq t \leq 1$. (3 pts)

Problem 4 (5 pts). A heterogeneous reaction is known to occur at a rate described by the following expression:

$$log_{10}(r) = log_{10}(k_1 P_A) - 2log_{10}(1 + k_A P_A + k_B P_B), \tag{6}$$

where log_{10} is the log base 10, and $k_1 = 0.015$. Using an optimization method and the following rate data, estimate the values of k_A and k_B.

P_A	P_B	r
1	0	3.40E-05
0.9	0.1	3.60E-05
0.8	0.2	3.70E-05
0.7	0.3	3.90E-05
0.6	0.4	4.00E-05

Table 1: Rate data for different values of P_A and P_B.

Solutions

Problem 1c:

```
Sub Newton()

Dim maxiter, iter, row As Integer
Dim tol, err As Double
Dim a, b, c, d, x0, x, f, fprime As Double

'Make the current Worksheet the selected one:
Worksheets(ActiveSheet.Name).Activate

'Clear up the page
Range("A7:B10").Clear
Range("E2:F20").Clear

maxiter = 100          'Maximum number of iterations

a = Cells(1, 1)
b = Cells(2, 1)
c = Cells(3, 1)
d = Cells(4, 1)
tol = Cells(5, 1)
x0 = Cells(6, 1)

Cells(1, 5) = "'iteration"
Cells(1, 6) = "'error"

err = 1
iter = 0
x = x0
row = 1
While (Abs(err) > tol And iter < maxiter)
  iter = iter + 1
  Call Fcalc(a, b, c, d, x, f, fprime)
  err = f / fprime
  x = x - err
  row = row + 1
  Cells(row, 5) = iter
  Cells(row, 6) = err
Wend

If (iter >= maxiter) Then
    Cells(6, 1) = "Maximum iteration reached;"
    Cells(7, 1) = "Solution might not be valid"
Else
    Cells(7, 2) = "'=x"
    Cells(7, 1) = x
    Cells(8, 2) = "'=iterations"
    Cells(8, 1) = iter
End If

Worksheets(ActiveSheet.Name).Cells.Font.Size = 16
Worksheets(ActiveSheet.Name).Cells.Font.Name = "Arial"
Range("F2:F20").NumberFormat = "0.00E+00"
```

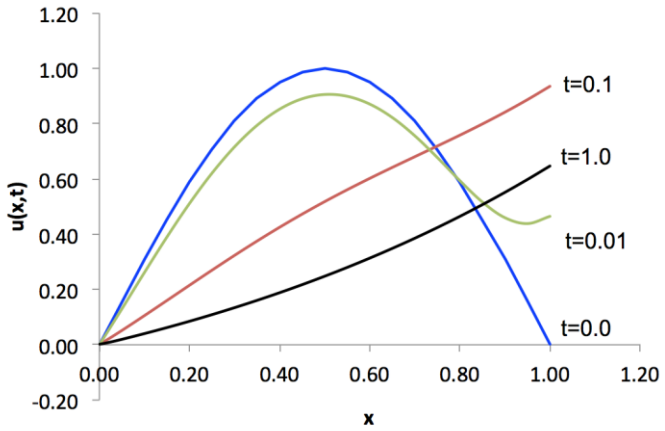
```
End Sub
Sub Fcalc(a, b, c, d, x, f, fprime)
  f = (a + b / x) - d / (x - c)
  fprime = -b / x ^ 2 + d / (x - c) ^ 2
End Sub
```

Problem 2:

```
Option Explicit
Option Base 1

Sub CrankNicholsonB()
Dim Bcoeff, Deltax, Deltat, Ratio As Double
Dim Numxsteps As Long
Dim Maxtime As Double
Dim Time As Double
Dim V() As Double
Dim U() As Double

Dim C() As Double
Dim D() As Double
Dim E() As Double
Dim B() As Double
Dim X() As Double
Dim i, j As Long

'clear the active sheet
Dim Lastrow As Long
Lastrow = Range("A" & Rows.Count).End(xlUp).Row
Range("A7:V" & Lastrow).Clear

Bcoeff = ActiveSheet.Cells(1, 3).Value  'get B coefficient
Deltat = ActiveSheet.Cells(2, 3).Value  'get time step
Deltax = ActiveSheet.Cells(3, 3).Value  'get distance step
Ratio = Deltax ^ 2 / Deltat

Maxtime = ActiveSheet.Cells(2, 5).Value 'get maximum time
Numxsteps = Round(1 / Deltax) - 1
Time = 0                        'initialize time

ReDim V(1 To Numxsteps + 2) As Double
ReDim U(1 To Numxsteps + 2) As Double

ReDim C(1 To Numxsteps) As Double       'lower diagonal coefficients
ReDim D(1 To Numxsteps) As Double       'diagonal coefficients
ReDim E(1 To Numxsteps) As Double       'upper diagonal coefficients
ReDim B(1 To Numxsteps) As Double       'right hand side vector
ReDim X(1 To Numxsteps) As Double       'unknown temperatures at new time step

'Get the initial temperatures from the spreadsheet

i = 6

For j = 1 To Numxsteps + 2
  V(j) = ActiveSheet.Cells(i, j + 1)
Next j
```

```
Time = 0
While Time <= Maxtime

'Set up the tridiagonal system coefficients

  For j = 2 To Numxsteps + 1
    C(j - 1) = -1 - Bcoeff * Deltax / 2
    D(j - 1) = 2 * (1 + Ratio)
    E(j - 1) = -1 + Bcoeff * Deltax / 2
    B(j - 1) = (1 + Bcoeff * Deltax / 2) * V(j - 1) - 2 * (1 - Ratio) * V(j) + (1 - Bcoeff * Deltax / 2) * V(j + 1)
  Next j

  'right boundary correction
  D(Numxsteps) = D(Numxsteps) + 4 / 3 * E(Numxsteps)
  C(Numxsteps) = C(Numxsteps) - 1 / 3 * E(Numxsteps)
  B(Numxsteps) = B(Numxsteps) - 2 * Deltax / 3 * E(Numxsteps)

  Call Thomas(Numxsteps, C, D, E, B, X)

  U(1) = 0
  For j = 1 To Numxsteps
    U(j + 1) = X(j)
  Next j
  U(Numxsteps + 2) = (4 / 3) * U(Numxsteps + 1) - (1 / 3) * U(Numxsteps) + 2 * Deltax / 3

  Time = Time + Deltat   'this is the "new" time

  'display the temperatures at the new time
  i = i + 1                 'move down one row
  ActiveSheet.Cells(i, 1) = Time
  For j = 1 To Numxsteps + 2
    ActiveSheet.Cells(i, j + 1) = U(j)
  Next j

  For j = 1 To Numxsteps + 2
    V(j) = U(j)
  Next j
Wend

'change worksheet font size and name
With ActiveSheet
      .Cells.Font.Size = "12"
      .Cells.Font.Name = "Arial"
End With

'set the number of decimal digits
Columns(1).NumberFormat = "0.000"
Range("B5:V" & Lastrow).NumberFormat = "0.00"

End Sub

Sub Thomas(N, C, D, E, B, X)
Dim Beta() As Double
Dim Gamma() As Double
ReDim Beta(1 To N) As Double
```

```
ReDim Gamma(1 To N) As Double
Dim i As Long
Dim j As Long

Beta(1) = D(1)
Gamma(1) = B(1) / Beta(1)
For i = 2 To N
  Beta(i) = D(i) - C(i) * E(i - 1) / Beta(i - 1)
  Gamma(i) = (B(i) - C(i) * Gamma(i - 1)) / Beta(i)
Next i
X(N) = Gamma(N)
For i = 2 To N
  j = N - i + 1
  X(j) = Gamma(j) - E(j) * X(j + 1) / Beta(j)
Next i

End Sub
```

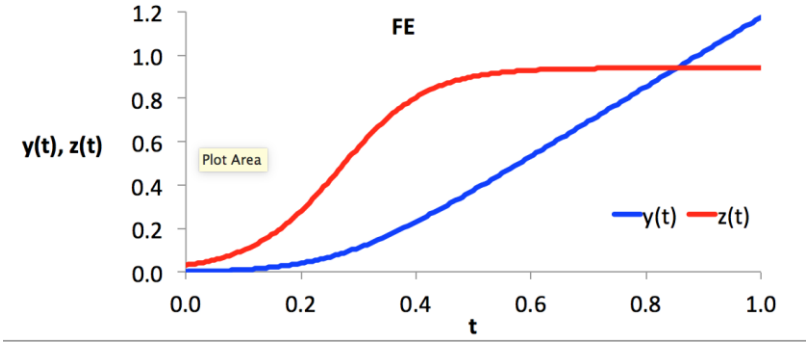


Problem 3a

```
Option Base 1
Option Explicit
Sub rk2and4()
'Main routince for solving a set of ODE's using the RK2 Method.
    Dim y() As Double               'size N
    Dim f() As Double               'size N
    Dim h As Double                 'fixed time step
    Dim b1, b2, b3 As Double                'intermediate time
    Dim TimeMax As Double       'maximum integration time
    Dim Time As Double              'time counter
    Dim ActRow As Integer           'row counter
    Dim i As Integer                'a counter
    Dim N As Integer                ' N = number of state variables and equations

    Dim w() As Double               'size N
    Dim k1(), k2(), k3(), k4() As Double  'size N

    Worksheets(ActiveSheet.Name).Activate

    'clear the worksheet
    Dim Lastrow As Long
```

```
Lastrow = Range("A" & Rows.Count).End(xlUp).Row
If Lastrow > 9 Then
  Range("E9:G" & Lastrow).Clear
End If

N = 2          'The number of ODEs
ReDim y(N) As Double                  'size N
ReDim f(N) As Double                  'size N
ReDim w(N) As Double                  'size N
ReDim k1(N), k2(N), k3(N), k4(N) As Double   'size N

TimeMax = ActiveSheet.Cells(1, 2).Value   'get TimMax from spreadsheet
h = ActiveSheet.Cells(2, 2).Value         'get DeltaT from spreadsheet
b1 = ActiveSheet.Cells(3, 2).Value      'get coefficient b1
b2 = ActiveSheet.Cells(4, 2).Value      'get coefficient b2
b3 = ActiveSheet.Cells(5, 2).Value      'get coefficient b3

For i = 1 To N
  y(i) = ActiveSheet.Cells(8, i + 5).Value 'get initial y values
Next i

ActRow = 9                            'calculated output starts here
Time = 0
Call FCalc(b1, b2, b3, y(), f())
While (Time <= TimeMax)
   '################################
   ' RK2
   For i = 1 To N
      k1(i) = h * f(i)
      w(i) = y(i) + k1(i)
   Next i
   Call FCalc(b1, b2, b3, w(), f())
   For i = 1 To N
      k2(i) = h * f(i)
      y(i) = y(i) + 1 / 2 * (k1(i) + k2(i))
   Next i
   '################################
   ' RK4
   'For i = 1 To N
   '    k1(i) = h * f(i)
   '    w(i) = y(i) + k1(i) / 2
   'Next i
   'Call FCalc(b1, b2, b3, w(), f())
   'For i = 1 To N
   '    k2(i) = h * f(i)
   '    w(i) = y(i) + k2(i) / 2
   'Next i
   'Call FCalc(b1, b2, b3, w(), f())
   'For i = 1 To N
   '    k3(i) = h * f(i)
   '    w(i) = y(i) + k3(i)
   'Next i
   'Call FCalc(b1, b2, b3, w(), f())
   'For i = 1 To N
   '    k4(i) = h * f(i)
   '    y(i) = y(i) + (k1(i) + 2 * k2(i) + 2 * k3(i) + k4(i)) / 6
```

```
    'Next i
    '################################

    Time = Time + h        'Display the results on the spreadsheet
    ActiveSheet.Cells(ActRow, 5) = Time      'put time on spreadsheet
    For i = 1 To N
       ActiveSheet.Cells(ActRow, i + 5).Value = y(i) 'output
    Next i
    ActRow = ActRow + 1
Wend                            'repeat for all times
'change the fontsize
Lastrow = Range("E" & Rows.Count).End(xlUp).Row
Range("E6:G" & Lastrow).Font.Size = 14
Range("E6:G" & Lastrow).Font.Name = "Arial"
Range("E8:E" & Lastrow).NumberFormat = "0.000"
Range("F8:G" & Lastrow).NumberFormat = "0.00"
End Sub
'###################################
Sub FCalc(b1, b2, b3, w, f)

  f(1) = b1 * w(1) - b1 / b2 * w(1) * w(1)
  f(2) = b3 * w(1)

 End Sub
'###################################
```



Problem 4

	A	B	C	D	E	F
1	kA=	19.94				
2	kB=	5.01				
3	k1=	0.015				
4						
5	P_A	P_B	r	rcalc	residual	residual^2
6	1	0	3.40E-05	3.42E-05	2.57E-03	0.000
7	0.9	0.1	3.60E-05	3.57E-05	-3.77E-03	0.000
8	0.8	0.2	3.70E-05	3.72E-05	2.55E-03	0.000
9	0.7	0.3	3.90E-05	3.87E-05	-2.90E-03	0.000
10	0.6	0.4	4.00E-05	4.02E-05	1.73E-03	0.000
11						
12	sum					0.000
13						